SAMBER/LO · GO CONF TALK

samber/lo goes vectorized.

SIMD in Go: from 4383ns → 89ns.

Samuel Berthe · @samber / github.com/samber/lo / 28 April 2026

$ whoami

Samuel.

Hi, I'm **Sam** — a Go dev from Nantes who spends too much time in open source.

> Ex-CTO @ **Screeb**

> Ex-Head of studies @ **Epitech**

> Organizer · **GenAI Nantes**, **Shift**

> Full-time open-sourcer

> **github.com/samber**

— THE BENCHMARK

```
lo.Sum(numbers)              →  4383 ns

losimd.SumInt8x64(numbers)   →  89 ns
```

49×

Same slice. Same result. Same goroutine.
Different instructions sent to the CPU.

SIMD = Single Instruction, Multiple Data.

One instruction touches many lanes in one clock cycle. The CPU doesn't loop.

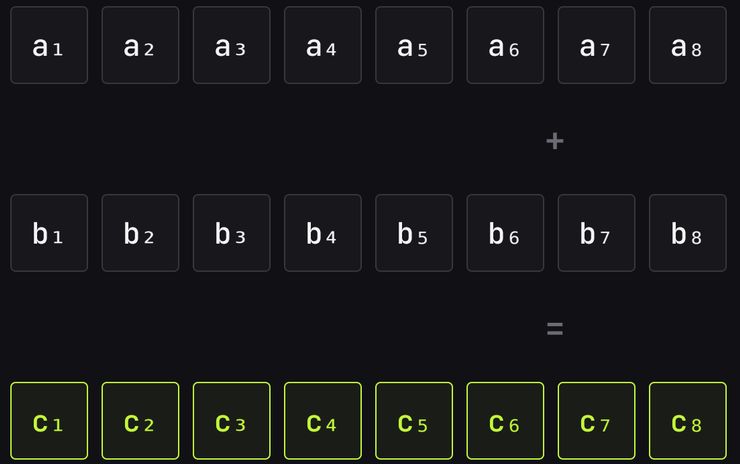
— SCALAR (WHAT GO DOES TODAY)

1 instruction → 1 result

1 cycle · **1 value**

— SIMD 512-BIT (AVX-512)

1 instruction → 8 results



1 cycle · **8 values**

Wider registers, more lanes per cycle.

Each generation doubled the width. Bigger register = more work per instruction.

EXTENSION	WIDTH	INT8	INT16	INT64	YEAR	AVAILABILITY
SSE2	128-bit	16	8	2	2001	universal on x86-64
AVX / AVX2	256-bit	32	16	4	2012 / 2113	EC2, GCP, mainstream cloud
AVX-512	512-bit	64	32	8	2017	Xeon, AMD Zen 4 (Hetzner AX)

Masks = per-lane predicate bits.

A comparison writes a k-register. The next instruction reads it as its gate. No branch.

— ZEROING {K1}{Z}

Inactive lanes → 0

src	1	2	3	4
mask	1	0	1	0
× 2	2	0	6	0

— MERGE {K1}

Inactive lanes → keep dst

dst	9	9	9	9
src × 2	2	—	6	—
mask	1	0	1	0
out	2	9	6	9

VPCMPGTD produces the mask. The next op reads it. The branch disappears.

One compare. One multiply. No branches.

scalar vs SIMD · pseudocode

```
// scalar — branches on every element
for i, v := range data {
    if v > 0 {
        result[i] = v * 2
    }
}

// SIMD equivalent
vec    = VMOVDQU(data[0:16])       // load 16×int32 into ZMM0
zero   = VPBROADCASTD(0)            // ZMM1 = [0, 0, …, 0]
k1     = VPCMPGTD(vec, zero)         // k1[i] = 1 if vec[i] > 0
result = VPMULLD(vec, 2) {k1}{z}    // multiply, gated by k1, else 0
```

— 02 / 03 —

Go finally catches up.

Who's been vectorized for years, what Go devs resorted to, and what Go 1.26 actually ships.

Systems built on SIMD.

You already ship vectorized code every day. Almost every hot path you depend on has a SIMD kernel underneath.

OLAP

Analytical databases

ClickHouse, **DuckDB**, **Snowflake**, **Umbra**, **Velox**, **Arrow**. Columnar scans, hash builds, aggregations, filters — vectorized execution is the architecture.

vectorized engines

CODECS

Media & compression

x264/**x265**, **dav1d**, **libjpeg-turbo**, **libvpx**, **zstd**, **lz4**, **brotli**. Pixel blocks, DCTs, entropy coding — naturally 8/16 lanes wide.

video · image · zip

ML

Inference & BLAS

llama.cpp, **onnxruntime**, **OpenBLAS**, **MKL**, **ggml**, **PyTorch CPU**. GEMM, dot products, int8 quantized matmul — every LLM token goes through vector lanes.

dot · GEMM · quant

PARSERS

Text & protocols

simdjson, **simdutf**, **ripgrep**, **Hyperscan**, **PCRE2-JIT**, HTTP/2 HPACK. Find delimiters, validate UTF-8, classify bytes — 64 chars per cycle.

JSON · UTF-8 · regex

CRYPTO

Hashing & ciphers

BLAKE3, **AES-NI**, **ChaCha20**, **SHA-NI**, **Poly1305**, **Argon2**. CPUs ship dedicated SIMD instructions — your entire TLS handshake lives here.

TLS · hashing · VPN

SEARCH

Vector DBs & ANN

FAISS, **Qdrant**, **Milvus**, **pgvector**, **Elasticsearch**, **HNSW**. L2, cosine, dot product over millions of embeddings — SIMD or die.

embeddings · kNN

GRAPHICS

Rendering & physics

Blender Cycles, **Embree**, **OSPRay**, **Unreal**, **Bullet**, **PhysX**. Ray/AABB intersection, matrix math, particle updates — all wide by nature.

ray tracing · simulation

AUDIO/DSP

Signal processing

FFTW, **PFFFT**, **libsamplerate**, **Web Audio**, **CoreAudio**. FFTs, convolutions, filters, resampling — the original SIMD workload.

FFT · conv · filters

INFRA

Networking & systems

DPDK, **VPP**, **Envoy**, **QUIC**, **XDP**, **CRC32C**. Packet classification, checksums, memcpy/memcmp — libc itself is SIMD under the hood.

packets · checksums

What Go devs did before.

Four workarounds. Each one leaves scars in your codebase.

— 01 · GENERATOR

Avo

Go-flavored assembly macro DSL. Emits plan9 `.s` files. The stdlib uses it for AES/SHA. Still assembly, just generated.

— 02 · CGO

cgo + intrinsics

C intrinsics via `immintrin.h`. **25–100ns overhead per call** — destroys gains for small N.

— 03 · ASM

Hand-written plan9

Raw `.s` files. No debugger support. Architecture-specific. Fragile.

— 04 · RUNTIME

purego + dlopen

Load native `.so` at runtime. Complex deployment. Version coupling headaches.

Go 1.26's native approach has none of this overhead.

Go 1.26 + GOEXPERIMENT=simd.

~/projects/hot-path $

```
$ go version
go version go1.26.0 linux/amd64

$ export GOEXPERIMENT=simd
$ go run main.go
// in main.go:
import "simd/archsimd"
```

— STATUS

Experimental

Opt-in per build. API may evolve.

— SCOPE

x86 only

SSE2 · AVX2 · AVX-512. ARM not yet.

— TRACK

golang/go#73787

pkg.go.dev/simd/archsimd

The pattern: Load → Operate → Store.

examples/add_int64.go

```go
import "simd/archsimd"

func AddInt64(a, b []int64) []int64 {
    vecA := archsimd.LoadInt64x8Slice(a)   // 8×int64 → ZMM
    vecB := archsimd.LoadInt64x8Slice(b)

    vecSum := vecA.Add(vecB)               // 1 instruction

    var buf [8]int64
    vecSum.Store(&buf)                     // back to memory
    return buf[:]
}
```

Explicit. No magic. You decide what enters a vector register, and when it leaves.
12 chunks of 8 + scalar tail of 4. lo/exp/simd handles the chunking for you.

lo/exp/simd — same ergonomics, 49× faster.

sum_example.go

```go
import (
    lo     "github.com/samber/lo"
    losimd "github.com/samber/lo/exp/simd"
)

numbers := randomInt8Generator(10_000)

// idiomatic lo, scalar
result1 := lo.Sum(numbers)                // 4383 ns

// SIMD + AVX-512
result2 := losimd.SumInt8x64(numbers)     //   89 ns
```

— NAMING

Op + Type + Lanes

SumInt8x64 · 64 lanes (512/8)

SumInt16x32 · 32 lanes (512/16)

SumInt32x16 · 16 lanes (512/32)

SumInt64x8 · 8 lanes (512/64)

SumFloat32x16

SumFloat64x8

Smaller type → more lanes → bigger win.

n = 10,000 elements · AVX-512 hardware · benchstat · warm cache

49×

— INT8

64 lanes per instruction

31×

— INT16

32 lanes per instruction

10×

— INT64

8 lanes per instruction

BENCHMARK	NS/OP	SPEEDUP
lo.Sum · int8	4 383	1×
losimd.SumInt8x64	89	49×
losimd.SumInt16x32	94	31×
losimd.SumInt64x8	187	10×

When SIMD wins.

The archetype is columnar analytics: 10M homogeneous values, one reduction.

Large N `N > 1 000`

Overhead amortized, register chunks fill completely. Rule of thumb before measuring.

Contiguous memory `[]T`

Slices and arrays. Not linked lists, not maps, not interface slices.

Pure operations `map · reduce · filter`

No side effects, no global mutation. Sum, min, max, transform.

Branch-free predicates `v > 0`

Simple comparisons become masks. Complex multi-field conditionals don't map.

Numeric types `int / float`

int8/16/32/64, float32/64. Not strings, not interfaces, not pointers.

Struct-of-arrays `SoA > AoS`

Packed, homogeneous, sequential. The data layout has to match the hardware model.

When SIMD loses.

Profile before assuming. Sometimes the compiler has already beaten you to it.

Small N `N < 100`

Register setup + chunking overhead dominates. Scalar often wins.

Memory-bound work

Waiting on RAM? Vector throughput doesn't help. SIMD is a compute optimization, not a memory one.

Branchy predicates

Complex business logic doesn't map to vector ops. Masks handle comparisons, not branches.

Non-contiguous access

Gather / scatter exist but are **slow** — often slower than scalar. The graph-traversal trap.

Already-vectorized output

Check with `go build -gcflags="-S" | grep VMOV`. You might already have SSE2.

Always benchmark

Both before and after. The "obvious win" sometimes regresses. Don't ship on vibes.

The CI problem: GitHub Actions says SIGILL.

Azure VMs strip the AVX-512 CPUID bit from guests. The instruction is valid — the hypervisor isn't.

recommended · runtime detection

```go
import "golang.org/x/sys/cpu"

func SumInt8x64(s []int8) int8 {
    if cpu.X86.HasAVX512F {
        return simdSum(s)
    }
    return scalarSum(s) // fallback
}
```

— DEFAULT GHA RUNNERS

Ubuntu on Azure · **no AVX-512** · SIGILL on any 512-bit op

— UBICLOUD

Bare-metal AVX-512 · Hetzner AX (Zen 4) · open-source AWS alternative

— SPLIT WORKFLOWS

Unit tests on GHA · benchmarks on a dedicated runner

— BUILD TAGS

`//go:build amd64 && avx512`

Hyperthreading ≠ more vector throughput.

SMT hides latency. SIMD has no latency to hide — the vector unit is already saturated.

	HORIZONTAL SCALING	UNIT OF SCALING	WHAT IT WINS YOU
SIMD	within a core	register width (lanes)	10–50× on numeric hot paths
SMT	within a core	latency hiding	10–40% · nothing on SIMD-heavy work
SMP	across cores	core count	true parallelism, N× if N cores

> If thread A hammers the vector ALU back-to-back, thread B on the sibling gets nothing.
> Some HPC shops **disable SMT** and pin one thread per physical core.

What's missing — come build it.

lo/exp/simd is experimental. The lowest-friction contribution is probably runtime CPU detection.

ARM NEON / SVE

Apple Silicon, Graviton, Ampere — all excluded today. Patterns from x86 translate.

Auto-vectorization

GCC and Clang do it at `-O2`. Go is conservative — explicit SIMD is the only path for now.

Alignment helpers

No way to request a 64-byte-aligned slice. Must use `unsafe` tricks today.

Runtime detection

Wrap functions, transparent scalar fallback. Makes it usable in production without the CI headache.

More operations

Filter, Map, Reduce variants. Min/Max. Dot product. The surface area is wide open.

Benchmarks + validation

Diverse hardware, diverse N. Real production workloads with actual data.

→ github.com/samber/lo → lo.samber.dev/docs/experimental/simd → github.com/sponsors/samber

Thank you !

*« Liberté, Égalité, **Opensourcé.** »*

$ github.com/samber/**talks** — SLIDES & DECKS

$ github.com/sponsors/**samber** — SUPPORT OSS

$ Built with **Claude Design**